PaXini

API Reference



PaXini Tech

GMH15



CONTENTS

PaXini

1. Control class DexH15Control ——— 01

2. Camera control class DexH15Camera ——— 37

3. Kinematic Analysis Class DexH15Kinematic ——— 40

1. Control class DexH15Control

is responsible for core operations such as serial communication, motor control, and sensor reading of the dexterous hand.

1.1 connectModbusAuto

- **Function Overview**

  Automatically connect to the first dexterous hand device under the current serial port. Suitable for single device scenario , the system will automatically complete the device discovery and connection process.

- **Function prototype**

```
1  bool connectModbusAuto(const std::string& hand_port_num, int16_t& device_address);
```

- **Parameter Description**

Parameter Name	Type	Input / Output	Description
hand_port_num	const std::string&	Input	Serial port number connected to the dexterous hand (e.g., "/dev/ttyUSB0")
device_address	int16_t&	Output	The first device address returned after a successful connection

- **Return value**

  true: Connection successful
  false: Connection failed

- **Special Instructions**

  If an error occurs during the execution of the program, an exception will be thrown.

1.2 openModbusDevice

- **Function Overview**

  Open the specified serial port device and establish a Modbus communication connection.

- **Function prototype**

```
1  bool openModbusDevice(const std::string& hand_port_num, int32_t baud_rate);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
hand_port_num	const std::string&	Input	Serial port number connected to the dexterous hand
baud_rate	int32_t	Input	Baud rate for serial device communication (4000000, 921600, 460800, 230400, 115200, 57600)

- **Return value**

  true: Open successful
  false: Failed to open

1.3 scanModbusDevices

- **Function Overview**

  Scan all communicable device addresses under the opened serial port.

- **Function prototype**

```
1  std::vector<int16_t> scanModbusDevices(const std::string& hand_port_num);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
hand_port_num	const std::string&	Input	Serial port number connected to the dexterous hand

- **Return value**

  Returns the set of addresses of all communicable devices under the opened serial port (std::vector<int16_t>).

1.4 initModbusDevice

- **Function Overview**

  Initialize the Modbus device configuration of the dexterous hand.

- **Function prototype**

```
1  int initModbusDevice(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

  1: Initialization successful
  0: Initialization failed

1.5 disconnectModbus

- **Function Overview**

Close the current Modbus serial port connection and release the serial port resources.

- **Function prototype**

```
1  bool disconnectModbus();
```

- **Parameter Description**

None

- **Return value**

true: Disconnection successful
false: Disconnection failed

1.6 isModbusDeviceConnected

- **Function Overview**

Check the Modbus connection status of the dexterous hand at the specified slave address.

- **Function prototype**

```
1  bool isModbusDeviceConnected(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

true: Device is connected
false: Device not connected

1.7 setModbusBaudrate

- **Function Overview**

Set the communication baud rate of the Modbus slave station.

- **Function prototype**

```
1  bool setModbusBaudrate(int16_t slave_address, int32_t baud_rate);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
baud_rate	int32_t	Input	Target baud rate (4000000, 921600, 460800, 230400, 115200, 57600)

- **Return value**

true: Setting successful
false: Setting failed

- **Special Instructions**

  - The baud rate setting must be completed before using the serial port, and after the setting is completed the device needs to be restarted to take effect.
  - All devices connected to the same serial portmust maintain the same baud rate.

1.8 getModbusBaudrate

- **Function Overview**

Get the communication baud rate of the current Modbus slave station.

- **Function prototype**

```
1  int32_t getModbusBaudrate(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns the current baud rate value of the slave device (int32_t).

1.9 setModbusDeviceAddress

- **Function Overview**

Modify the device address of the Modbus slave station.

- **Function prototype**

```
bool setModbusDeviceAddress(int16_t slave_address, int16_t new_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Current slave address (0x01 ~ 0xF7)
new_address	int16_t	Input	New slave address (0x01 ~ 0xF7)

- **Return value**

true: Setting successful
false: Setting failed

- **Special Instructions**

The setting of the slave address must be completed before use. After the setting is completed, the device needs to be restarted to take effect.

1.10 openUsbDevice

- **Function Overview**

Open the USB serial device and establish a connection, then automatically obtain the slave address.

- **Function prototype**

```
bool openUsbDevice(const std::string& hand_port_num, int16_t& device_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
hand_port_num	const std::string&	Input	USB Device Number
device_address	int16_t&	Output	Slave address returned after successful connection

- **Return value**

true: Open successful
false: Open failed

1.11 disconnectUsb

- **Function Overview**

Close the USB device connection and release related resources.

- **Function prototype**

```
bool disconnectUsb();
```

- **Parameter Description**

None

- **Return value**

true: Operation execution completed

1.12 setMotorControlMode

- **Function Overview**

Set the control mode of the motor. The default mode is **position mode**, which can be switched to current mode or speed mode as needed.

- **Function prototype**

```
bool setMotorControlMode(int16_t slave_address, const ControlMode mode);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
mode	const ControlMode	Input	Motor control mode enumeration value

- **Explanation of ControlMode Enumeration Values**

Enumerated Value	Value	Description
CURRENT_CONTROL_MODE	1	Current Loop Control Mode
VELOCITY_CONTROL_MODE	2	Speed loop control mode
POSITION_CONTROL_MODE	3	Position Loop Control Mode (Default)

- **Return value**

true: Setting successful
false: Setting failed

1.13 getHandType

- **Function Overview**

Query the left or right hand type of the dexterous hand.

- **Function prototype**

```
1  HandType getHandType(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns HandType enum value, indicating the left or right hand type of the dexterous hand.

Enumerated Value	Value	Description
NONE_HAND	0	Invalid dexterous hand type
LEFT_HAND	1	Left Hand
RIGHT_HAND	2	Right Hand

1.14 setMotorMaxCurrent

- **Function Overview**

Set the maximum current for all motors to limit the peak grip force.

- **Function prototype**

```
1  bool setMotorMaxCurrent(int16_t slave_address, const Dex15MotorCurrent& max_current);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
max_current	const Dex15MotorCurrent&	Input	Maximum current configuration for each motor, with a value range of [0, 1000]

- **Return value**

true: Setting successful
false: Setting failed

1.15 getMotorMaxCurrentConfig

- **Function Overview**

Get the maximum current configuration of all motors in the current slave device.

- **Function prototype**

```
1  Dex15MotorCurrent getMotorMaxCurrentConfig(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns the Dex15MotorCurrent structure, which contains the maximum current configuration for all motors.

1.16 setMotorMaxVelocity

- **Function Overview**

Set the maximum speed of all motors.

- **Function prototype**

```
1  bool setMotorMaxVelocity(int16_t slave_address, const Dex15MotorSpeed& max_speed);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
max_speed	const Dex15MotorSpeed&	Input	Maximum speed configuration for each motor, with a value range of [0, 1000]

- **Return value**

true: Setting successful
false: Setting failed

1.17 getMotorMaxVelocityConfig

- **Function Overview**

Get the maximum speed configuration of all motors of the current slave device.

- **Function prototype**

```
1  Dex15MotorSpeed getMotorMaxVelocityConfig(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns Dex15MotorSpeed structure, which contains the maximum speed configuration for all motors.

1.18 enableMotor

- **Function Overview**

Enable (power on and activate) all motors of the specified slave station.

- **Function prototype**

```
bool enableMotor(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

true: Enable success
false: Enable failure

1.19 disableMotor

- **Function Overview**

Disable (power off) all motors of the specified slave station.

- **Function prototype**

```
bool disableMotor(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

true: Lower enable successful
false: Lower enable failure

1.20 isMotorEnabled

- **Function Overview**

Query the current enable status of the specified slave motor.

- **Function prototype**

```
bool isMotorEnabled(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

true: Motor is enabled
false: Motor is not enabled

1.21 setJointPositionsAngle (All Joints)

- **Function Overview**

Control the target angles of all 11 finger joints (normalized angles).

- **Function prototype**

```
int setJointPositionsAngle(int16_t slave_address, const std::vector<double>& joint_angles);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
joint_angles	const std::vector<double>&	Input	Set of normalized angle values for 11 adjustable joints

- **Description of the joint_angles element**

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	corresponds to the finger	Value Range	Description
0	index_joint1	index finger	[0, 1000]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 1000]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 1000]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 1000]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 1000]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 1000]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 1000]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 1000]	End joint, bending and extension
8	thumb_joint1	Thumb	[-500, 500]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 1000]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 1000]	End joint, bending and extension

- **Return value**

1: Setup successful
0: Setting failed

1.22 setJointPositionsAngle (Specify Finger)

• Function Overview

Set the normalized joint angle of a specified single finger.

• Function prototype

```
int setJointPositionsAngle(int16_t slave_address, FingerType finger_id, const std::vector<double>& joint_angles);
```

• Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
finger_id	FingerType	Input	Enumeration value of the target finger
joint_angles	const std::vector<double>&	Input	Set of normalized angle values for the specified finger

• Definition of FingerType Enum Values

Enumerated Value	Value	Description
INVALID_FINGER	0	Invalid value
INDEX_FINGER	1	index finger
MIDDLE_FINGER	2	middle finger
RING_FINGER	3	ring finger
PINKY_FINGER	4	Little Finger
THUMB_FINGER	5	Thumb

• Return value

1: Setup successful
0: Setting failed

1.23 calculateRealAngle (Control Class)

• Function Overview

Calculate the corresponding real physical angle (unit: degrees) based on the normalized angle value.

• Function prototype

```
int calculateRealAngle(int16_t slave_address,
                       const std::vector<double>& normalize_angle,
                       std::vector<double>& real_angle);
```

• Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
normalize_angle	const std::vector<double>&	Input	Set of normalized angle values for 11 adjustable joints
real_angle	std::vector<double>&	Output	The set of 11 calculated real angle values of joints

• Description of normalize_angle element

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).



Index	Joint Identification	corresponds to the finger	Value Range	Description
0	index_joint1	index finger	[0, 1000]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 1000]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 1000]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 1000]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 1000]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 1000]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 1000]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 1000]	End joint, bending and extension
8	thumb_joint1	Thumb	[-500, 500]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 1000]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 1000]	End joint, bending and extension

• Return value

1: Calculation successful
0: Calculation failed

1.24 calculateNormalizeAngle (Control Class)

- **Function Overview**

Calculate the corresponding normalized angle based on the real physical angle value.

- **Function prototype**

```
int calculateNormalizeAngle(int16_t slave_address,
                            const std::vector<double>& real_angle,
                            std::vector<double>& normalize_angle);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
real_angle	const std::vector<double>&	Input	Set of real angle values (unit: degrees) for 11 adjustable joints
normalize_angle	std::vector<double>&	Output	The calculated set of normalized angle values for 11 adjustable joints

- **Description of real_angle element**

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	corresponds to the finger	Value Range	Description
0	index_joint1	index finger	[0, 80]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 80]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 80]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 80]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 80]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 80]	End joint, bending and extension
6	little_joint1	Little finger	[0, 80]	Proximal joint, flexion and extension
7	little_joint2	Little finger	[0, 80]	End joint, bending and extension
8	thumb_joint1	Thumb	[0, 109.6]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 87]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 120.1]	End joint, bending and extension

- **Return value**

1: Calculation successful
0: Calculation failed

1.25 setMotorTargetCurrent (All Motors)

- **Function Overview**

In **current mode** , set the target current for all motors. The larger the target current value, the stronger the hand grip force.

- **Function prototype**

```
int setMotorTargetCurrent(int16_t slave_address, int16_t target_current);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
target_current	int16_t	Input	Unified target current for all motors, with a value range of [-1000, 1000]

- **Return value**

1: Setup successful
0: Setting failed

1.26 setMotorTargetCurrent (specified motor)

- **Function Overview**

In **current mode** , set the target current for the specified single motor.

- **Function prototype**

```
int setMotorTargetCurrent(int16_t slave_address,
                          FingerMotor motor_id,
                          int16_t target_current);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
motor_id	FingerMotor	Input	Enumerated index value of the target motor
target_current	int16_t	Input	Target current value, range [-1000, 1000]

- **Definition of FingerMotor Enumeration Values**

Enumerated Value	Value	Description
INVALID_MOTOR	0	Invalid value
INDEX_MOTOR	1	Index finger motor
MIDDLE_MOTOR	2	Middle finger motor
RING_MOTOR	3	Ring finger motor
PINKY_MOTOR	4	Little Finger Motor
THUMB_MOTOR_1	5	Thumb side swing motor
THUMB_MOTOR_2	6	Thumb Motor 2
THUMB_MOTOR_3	7	Thumb Motor 3

- **Return value**

1: Setup successful
0: Setting failed

1.27 getMotorActualCurrent

- **Function Overview**

Get the actual current values of all current motors.

- **Function prototype**

```
int getMotorActualCurrent(int16_t slave_address, Dex15MotorCurrent& actual_current);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
actual_current	Dex15MotorCurrent&	Output	Contains the actual current of all motors , ranging from [-1000, 1000]

- **Return value**

1: Acquisition Successful
0: Acquisition failed

1.28 setMotorTargetVelocity (All Motors)

- **Function Overview**

Under**Speed Mode**, the target speed of all motors is uniformly set to control the overall movement speed of the hand.

- **Function prototype**

```
int setMotorTargetVelocity(int16_t slave_address, int16_t target_speed);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
target_speed	int16_t	Input	The unified target speed for all 7 motors, with a value range of [-1000, 1000]

- **Return value**

1: Setup successful
0: Setting failed

1.29 setMotorTargetVelocity (specify motor)

- **Function Overview**

Under **speed control mode** , set the target speed of the specified single motor.

- **Function prototype**

```
int setMotorTargetVelocity(int16_t slave_address, FingerMotor motor_id, int16_t target_speed);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
motor_id	FingerMotor	Input	Enumerated index value of the target motor
target_speed	int16_t	Input	Target speed value, range [-1000, 1000]

- **Definition of FingerMotor Enumeration Values**

Enumerated Value	Value	Description
INVALID_MOTOR	0	Invalid value
INDEX_MOTOR	1	Index finger motor
MIDDLE_MOTOR	2	Middle finger motor
RING_MOTOR	3	Ring finger motor
PINKY_MOTOR	4	Little Finger Motor
THUMB_MOTOR_1	5	Thumb side swing motor
THUMB_MOTOR_2	6	Thumb Motor 2
THUMB_MOTOR_3	7	Thumb Motor 3

- **Return value**

1: Setup successful
0: Setting failed

1.30 getMotorActualVelocity

- **Function Overview**

Get the actual operating speeds of all current motors.

- **Function prototype**

```
int getMotorActualVelocity(int16_t slave_address, Dex15MotorSpeed& actual_speed);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
actual_speed	Dex15MotorSpeed&	Output	Contains the actual speed of all motors , ranging from [-1000, 1000]

- **Return value**

1: Success in Acquisition
0: Acquisition failed

1.31 calibrateSensor

- **Function Overview**

Calibration of tactile sensors.

- **Function prototype**

```
bool calibrateSensor(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

true: Calibration successful
false: Calibration failed

1.32 getJointPositionsAngle (all joints)

- **Function Overview**

Obtain the current normalized angles of the finger joints. When is_all_joint is false, 11 adjustable joint angles are acquired; when it is true, 4 additional distal interphalangeal (DIP) joints are returned, resulting in a total of 15 joint angles being obtained.

- **function prototype**

```
int getJointPositionsAngle(int16_t slave_address, std::vector<double>& joint_angles, bool is_all_joint = false);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
joint_angles	std::vector<double>&	Output	Set of normalized joint angle values, the number of elements is determined by is_all_joint
is_all_joint	bool	Input	Whether to obtain all joint angles, default is false (11 adjustable joints are obtained)

When is_all_joint = false, description of joint_angles elements (11 in total, in the order: 2 for index finger → 2 for middle finger → 2 for ring finger → 2 for little finger, and 3 for thumb)

Index	Joint Identification	Corresponding finger	Value Range	Description
0	index_joint1	index finger	[0, 1000]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 1000]	Distal joint, flexion and extension
2	middle_joint1	middle finger	[0, 1000]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 1000]	Distal joint, flexion and extension
4	ring_joint1	ring finger	[0, 1000]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 1000]	Distal joint, flexion and extension
6	little_joint1	little finger	[0, 1000]	Proximal joint, flexion and extension
7	little_joint2	little finger	[0, 1000]	Distal joint, flexion and extension
8	thumb_joint1	thumb	[-500, 500]	Metacarpophalangeal biaxial joint, lateral oscillation
9	thumb_joint2	thumb	[0, 1000]	Proximal joint, flexion and extension
10	thumb_joint3	thumb	[0, 1000]	Distal joint, flexion and extension

When is_all_joint = true, description of joint_angles elements (15 in total, with 1 additional DIP joint of the distal phalanx for each of the four fingers)

Index	Joint Identification	Corresponding finger	Value Range	Description
0	index_joint1	index finger	[0, 1000]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 1000]	Mid-joint, flexion and extension
2	index_joint3	index finger	[0, 1000]	Distal interphalangeal joint, flexion and extension
3	middle_joint1	China Index Academy	[0, 1000]	Proximal joint, flexion and extension
4	middle_joint2	China Index Academy	[0, 1000]	Mid-joint, flexion and extension
5	middle_joint3	China Index Academy	[0, 1000]	Distal interphalangeal joint, flexion and extension
6	ring_joint1	ring finger	[0, 1000]	Proximal joint, flexion and extension
7	ring_joint2	ring finger	[0, 1000]	Mid-joint, flexion and extension
8	ring_joint3	ring finger	[0, 1000]	Distal interphalangeal joint, flexion and extension
9	little_joint1	little finger	[0, 1000]	Proximal joint, flexion and extension
10	little_joint2	little finger	[0, 1000]	Mid-joint, flexion and extension
11	little_joint3	little finger	[0, 1000]	Distal interphalangeal joint, flexion and extension
12	thumb_joint1	thumb	[-500, 500]	Metacarpophalangeal biaxial joint, lateral oscillation
13	thumb_joint2	thumb	[0, 1000]	Proximal joint, flexion and extension
14	thumb_joint3	thumb	[0, 1000]	Distal joint, flexion and extension

- **return value**

- 1: Acquisition successful
- 0: Acquisition failed

1.33 getJointPositionsAngle (specified finger)

- **Function Overview**

Get the current joint angles of the specified single finger. When is_all_joint is false, this finger returns 2 joints (3 for the thumb); when it is true, this finger additionally returns 1 distal phalanx (DIP) joint.

- **function prototype**

```
int getJointPositionsAngle(int16_t slave_address, FingerType finger_id, std::vector<double>& joint_angles,
                           bool is_all_joint = false);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
finger_id	FingerType	Input	Enumeration value of the target finger
joint_angles	std::vector<double>&	Output	The set of joint angle values of the specified fingers, the number of elements of which is determined by is_all_joint
is_all_joint	bool	Input	Whether to obtain all joint angles of this finger, default is false

- **return value**

1: Acquisition successful
0: Acquisition failed

1.34 getMotorControlMode

- **Function Overview**

Get the control mode of the current motor.

- **Function prototype**

```
ControlMode getMotorControlMode(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns ControlMode Enum value:

Parameter Name	Type	Input/Output
NONE	0	Invalid value (only appears when retrieval fails)
CURRENT_CONTROL_MODE	1	Current Loop Control Mode
VELOCITY_CONTROL_MODE	2	Speed loop control mode
POSITION_CONTROL_MODE	3	Position loop control mode

1.35 getFingerTactile[deprecated]

- **Function Overview**

Obtain the **resultant force** information of all finger tactile sensors. The hand has a total of 5 sensors, each returning a three-axis resultant force.

- **Function prototype**

```
1    std::vector<ForcePoint> getFingerTactile(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns a list containing 5 ForcePoint instances, each ForcePoint containing:

Field	Type	Description
x	float	Sensor X-axis force component, unit 0.1N
y	float	Sensor Y-axis force component, unit 0.1N
z	float	Sensor Z-axis force component, unit 0.1N

1.36 getFingerResultantForce

- **Function Overview**

Retrieves the resultant force data from all finger tactile sensors. The hand has 5 sensors in total, and each sensor returns a three-axis resultant force.

- **Function prototype**

```
1    std::vector<ForcePoint> getFingerResultantForce(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns a list containing 5 ForcePoint elements. Each ForcePoint contains:

Field	Type	Description
x	float	Sensor X-axis force component, unit 0.1N
y	float	Sensor Y-axis force component, unit 0.1N
z	float	Sensor Z-axis force component, unit 0.1N

1.37 getFingerTactileDetail [deprecated]

- **Function Overview**

Obtain the **distributed force** detailed information of all finger tactile sensors. The hand has a total of 5 sensors, with a total of 327 distributed force points.

- **Function prototype**

```
1    std::vector<ForcePoint> getFingerTactileDetail(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns a list containing 327 ForcePoint . The distribution rules are as follows:

Parameter Name	Type
index finger	50
middle finger	50
ring finger	50
Little Finger	50
Thumb	127

Each ForcePoint contains x , y , z three-axis components, with a unit of 0.1N.

1.38 getFingerDistributedForce

- **Function Overview**

Retrieves detailed distributed force data from all finger tactile sensors. The hand has 5 sensors in total, providing 327 distributed force points.

- **Function prototype**

```
1    std::vector<ForcePoint> getFingerDistributedForce(int16_t slave_address);
```

- **Parameters**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns a list containing 327 ForcePoint elements, distributed as follows:

Parameter Name	Type
index finger	50
middle finger	50
ring finger	50
Little Finger	50
Thumb	127

Each ForcePoint contains x, y, z three-axis components, with a unit of 0.1N.

1.39 getTemperature

- **Function Overview**

Obtain the temperature data of all finger sensors. The hand has a total of 5 sensors, returning 5 temperature readings.

- **Function prototype**

```
std::vector<FingerTemp> getTemperature(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns a list containing 5 FingerTemp instances, each FingerTemp containing:

Field	Type	Input/Output
finger_name	std::string	Finger Name
sensor_ID	int	Sensor ID
temperature	float	Sensor Temperature Value (1° C)

1.40 getFingerTactileData

- **Function Overview**

Obtain in one go **the resultant force, distributed force, and temperature data of the specified single finger .**

- **Function prototype**

```
void getFingerTactileData(int16_t slave_address,
                          FingerType finger_id,
                          ForcePoint& finger_tac,
                          std::vector<ForcePoint>& finger_tac_detail,
                          FingerTemp& temperature);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
finger_id	FingerType	Input	Enumeration type value of the target finger
finger_tac	ForcePoint&	Output	Resultant force information of the tactile sensors of this finger
finger_tac_detail	std::vector<ForcePoint>&	Output	Distribution force information of the tactile sensors on this finger
temperature	FingerTemp&	Output	The sensor temperature data of this finger

- **Definition of FingerType Enum Values**

Enumerated Value	Value	Description
INVALID_FINGER	0	Invalid value
INDEX_FINGER	1	index finger
MIDDLE_FINGER	2	middle finger
RING_FINGER	3	ring finger
PINKY_FINGER	4	Little Finger
THUMB_FINGER	5	Thumb

- **Return value**

No return value (void).

1.41 getAllTactileData

- **Function Overview**

Obtain all at once **the resultant force, distributed force, and temperature data of all fingers .**

- **Function prototype**

```
void getAllTactileData(int16_t slave_address,
                       std::vector<ForcePoint>& finger_tac,
                       std::vector<ForcePoint>& finger_tac_detail,
                       std::vector<FingerTemp>& temperature);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
finger_tac	std::vector<ForcePoint>&	Output	Force information of the tactile sensors of all fingers combined
finger_tac_detail	std::vector<ForcePoint>&	Output	Tactile sensor distributed force information for all fingers
temperature	std::vector<FingerTemp>&	Output	Sensor temperature data for all fingers
temperature	FingerTemp&	Output	The sensor temperature data of this finger

- **Return value**

No return value (void).

1.42 checkHandStatus

- **Function Overview**

Check the overall operating status of the dexterous hand.

- **Function prototype**

```
int checkHandStatus(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Parameter Name	Type
1	Normal operation
2	Fault-tolerant operation state (an error has occurred, but it does not affect the use of core interfaces)
3	Non - operational state

1.43 getFaultCode

- **Function Overview**

Obtain the fault code of the current motor to diagnose the cause of motor failure.

- **Function prototype**

```
1  std::vector<FingerMotorsError> getFaultCode(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns std::vector<FingerMotorsError> fault code list, containing the current fault information for each motor.

Error Name	ErrorCode	Error Description	Explanation	Solution
OVER_VOLT	0	Finger motor overvoltage	The drive supply voltage is higher than 15V and lasts for 2 seconds	After the voltage is restored, simply execute the clear error command
UNDER_VOLT	1	Finger motor under voltage	The drive supply voltage is below 10V and lasts for 2 seconds	After the voltage is restored, simply execute the clear error command
OVER_CURRENTU	2	Finger motor U-phase overcurrent	U-phase current is too large	After the current returns to normal, simply execute the error clearing command
OVER_CURRENTY	3	Finger motor V-phase overcurrent	Excessive V-phase current	After the current returns to normal, simply execute the error clearing command
OVER_CURRENTW	4	Finger motor W-phase overcurrent	W-phase current is too large	After the current returns to normal, simply execute the error clearing command
OVER_TEMPERATURE	5	Finger motor overheating	Motor temperature is too high	After waiting for the finger motor temperature to return to the normal range, simply execute the error clearing command.
ADC_CALIB	6	Finger motor hardware self-check error	AD Static Deviation +10%	1. It is recommended to check the magnetic field environment of the driver attachment. After eliminating interference, simply power off and restart. 2. After performing Inspection 1, the issue still cannot be resolved. It is recommended to replace the driver board.
FOC_ERR	7	Finger motor FOC calculation error	None	None
OPD_IQ	8	Finger motor IQ overcurrent	Motor IQ current is too high	After the overcurrent of the IQ current is eliminated, simply execute the error clearing operation.
OPD_U	9	Finger motor U-phase open circuit	Detection performed during power-on initialization	It is recommended to check whether the U-phase wire of the motor is disconnected. After restoration, simply execute the error clearing operation.
OPD_V	10	Phase V open circuit fault in finger motor	Detection performed during power-on initialization	It is recommended to check whether the V-phase wire of the motor is disconnected. After restoration, simply execute the error clearing operation.
OPD_W	11	Finger motor W-phase open circuit	Detection performed during power-on initialization	It is recommended to check whether the W-phase wire of the motor is disconnected. After restoration, simply execute the error clearing operation.
POS_REF_OVER	12	Finger motor position exceeds upper limit	None	None
POS_REF_UNDER	13	Finger motor position is below the lower limit	None	None
ABZ_ERR	14	Finger motor encoder anomaly	Magnetic encoding cycle loss count greater than 0.25%	The encoder at the motor end has lost data. The encoder needs to be checked. After powering off and restarting, observe the phenomenon. If the error persists, it needs to be returned to the factory.
ABZ_Z_ERR	15	Finger motor encoder Z-phase anomaly	Z phase cannot be found during initialization	Encoder Z-phase error at the motor end. The encoder needs to be checked. After powering off and restarting, observe the phenomenon. If the error persists, it needs to be returned to the factory.
ABZ_CALIB_ERR	16	Encoder calibration error	Encoder cable not properly connected or not calibrated	Need to troubleshoot the encoder or wiring
SPEED_REF_OVER	17	Given speed limit exceeded	None	None
CURRENT_REF_OVER	18	Given current exceeds limit	None	None
JOINT_ENC_DISCONNECT	19	Magnetic encoder disconnection on the electric drive board	None	None
NORM_POS_NEGA_WARN	29	Normalizationmotorstroke appearssmallnegative number	Motor stroke value is less than 0 and greater than -20	None
NORM_POS_NEGA_ERR	30	Normalizedmotorstroke showslargenegative number	Motor stroke value is less than -20	None
DISCONNECT	31	Finger motor not connected	Communication between the finger drive and the control board is disconnected	After connection, the system will automatically clear errors and recover

1.44 clearFaultCode

- **Function Overview**

Clear the fault code of the current motor.

- **Function prototype**

```
1  bool clearFaultCode(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

true: Clear successful
false: Clear failed

- **Special Instructions**

Not all fault codes can be cleared. For details, please refer to the SDK's accompanying documentation.

1.45 getFirmwareVersion

- **Function Overview**

Obtain the firmware version information of the device.

- **Function prototype**

```
1  const FirmwareVersion getFirmwareVersion(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns the FirmwareVersion structure, which contains detailed information about the device's firmware version.

1.46 getSDKVersion

- **Function Overview**

Get the version number of the current SDK.

- **Function prototype**

```
const std::string getSDKVersion();
```

- **Parameter Description**

None

- **Return value**

Returns the SDK version number string (std::string).

1.47 initMotorPosition

- **Function Overview**

Initialize the positions of all motors to return the dexterous hand to its initial zero position state.

- **Function prototype**

```
int initMotorPosition(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

1: Initialization successful
0: Initialization failed

- **Special Instructions**

It is recommended to cal **immediately after the device is powered on** , with a maximum execution time of approximately **5 seconds** .

1.48 setMotorTargetPosition (All Motors)

- **Function Overview**

Unifiedly set the target positions of all motors through the motor position parameter structure.

- **Function prototype**

```
int setMotorTargetPosition(int16_t slave_address,
                           const Dex15MotorPosition& target_position);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
target_position	const Dex15MotorPosition&	Input	Target position values of all motors

- **Return value**

1: Setup successful
0: Setting failed

1.49 setMotorTargetPosition (specify motor)

- **Function Overview**

Set the target position of the specified single motor.

- **Function prototype**

```
int setMotorTargetPosition(int16_t slave_address,
                           FingerMotor motor_id,
                           int16_t target_position);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
motor_id	FingerMotor	Input	Enumerated index value of the target motor
target_position	int16_t	Input	Target motor position value

- **Definition of FingerMotor Enumeration Values**

Enumerated Value	Value	Description
INVALID_MOTOR	0	Invalid value
INDEX_MOTOR	1	Index finger motor
MIDDLE_MOTOR	2	Middle finger motor
RING_MOTOR	3	Ring finger motor
PINKY_MOTOR	4	Little Finger Motor
THUMB_MOTOR_1	5	Thumb side swing motor
THUMB_MOTOR_2	6	Thumb Motor 2
THUMB_MOTOR_3	7	Thumb Motor 3

- **Return value**

1: Setup successful
0: Setting failed

1.50 getMotorPosition

- **Function Overview**

Get the position information of all current motors.

- **Function prototype**

```
const Dex15MotorPosition getMotorPosition(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns Dex15MotorPosition structure, which contains the current position information of all motors.

1.51 getPressPositionForce

- **Function Overview**

Get the set of position force points of all current finger pressing forces, with the unit being 0.1N.

- **Function prototype**

```
std::vector<PressForcePoint> getPressPositionForce(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returnsstd::vector<PressForcePoint>, containing the press force position point data for all fingers, with the unit of 0.1N.

1.52 getDeviceSN

- **Function Overview**

Obtain the SN serial number of the device.

- **Function prototype**

```
std::string getDeviceSN(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns the device SN code string.

1.53 getHardwareVersion

- **Function Overview**

Get the hardware version information of the device.

- **Function prototype**

```
HardwareVersion getHardwareVersion(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns HardwareVersion structure, which contains detailed information about the device's hardware version.

1.54 getModbusConfig

- **Function Overview**

Obtain the Modbus communication configuration information of the current slave device.

- **Function prototype**

```
ModbusConfig getModbusConfig(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns ModbusConfig structure, which contains the current Modbus communication configuration.

1.55 getDeviceInfo

- **Function Overview**

Unify the acquisition of all device information, such as the SN code, hardware version, firmware version, and Modbus configuration, of the slave device through a single call.

- **Function prototype**

```
bool getDeviceInfo(int16_t slave_address,
                   std::string& sn,
                   HardwareVersion& hardware_version,
                   FirmwareVersion& firmware_version,
                   ModbusConfig& modbus_config);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
sn	std::string&	Output	Device SN Code
hardware_version	HardwareVersion&	Output	Device hardware version information
firmware_version	FirmwareVersion&	Output	Device firmware version information
modbus_config	ModbusConfig&	Output	Device Modbus communication configuration information

- **Return value**

true: Acquisition successful
false: Acquisition failed

1.56 getJointMagneticEncoder

- **Function Overview**

Obtain the raw data of the magnetic encoders of all joints of the current slave device.

- **Function prototype**

```
std::vector<int16_t> getJointMagneticEncoder(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **Return value**

Returns a list containing the values of all joint magnetic encoders (std::vector<int16_t>).

1.57getActionPosition

- **Function Overview**

Load and read the motor position information corresponding to the built-in or custom action sequence.

- **Function prototype**

```
int getActionPosition(int16_t slave_address,
                      int16_t index,
                      Dex15MotorPosition& action_position,
                      bool is_customized = true);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Action Sequence Number (range [1, 20] when is_customized=true; is_customized=tRange [1, 10] when false)
action_position	Dex15MotorPosition&	Output	Acquired motor position data
is_customized	bool	Input	true: Custom action; false: Official built-in action (default true)

- **Return value**

1: Success in Acquisition
0: Acquisition failed

1.58 runAction

- **Function Overview**

Execute the built-in or custom action with the specified number.

- **Function prototype**

```
int runAction(int16_t slave_address, int16_t index, bool is_customized = true);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Action Sequence Number (range [1, 20] when is_customized=true; is_customized=when false, the range is [1, 10]
is_customized	bool	Input	true: Custom action; false: Official built-in action (default true)

- **Return value**

1: Execution successful
0: Execution failed

1.59 saveCustomizedAction (Specify Action Frame)

- **Function Overview**

Save the specified motor position data as a user-defined action.

- **Function prototype**

```
int saveCustomizedAction(int16_t slave_address,
                         int16_t index,
                         const Dex15MotorPosition& action);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Custom action sequence number, range [1, 20]
action	const Dex15MotorPosition&	Input	Motor position data to be saved

- **Return value**

1: Save Successful
0: Save failed

1.60 saveCustomizedAction (current position)

- **Function Overview**

Save the current real-time position of the dexterous hand **directly as a custom action with the specified number .**

- **Function prototype**

```
int saveCustomizedAction(int16_t slave_address, int16_t index);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Custom action sequence number, range [1, 20]

- **Return value**

1: Save Successful
0: Save failed

1.61 saveCustomizedAction (Joint Angle)

- **Function Overview**

Calculate the motor position based on the specified set of joint normalized angles, and then save it as a custom action.

- **Function prototype**

```
int saveCustomizedAction(int16_t slave_address,
                         int16_t index,
                         std::vector<double> joint_angle_list);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Custom action sequence number, range [1, 20]
joint_angle_list	std::vector<double>	Input	Set of joint normalized angles to be saved (11 adjustable joints)

- **Return value**

1: Save Successful
0: Save failed

1.62 setCommonForceCtlParam

- **Function Overview**

Configure **the initial position and control parameters of the common force control** . After the configuration is completed, calculate the target command for each motion through computeCommonForceTargetCommand .

- **Function prototype**

```
int setCommonForceCtlParam(int16_t slave_address,
                           const std::vector<double>& joint_angles,
                           const ForceCtlParam& param);
```

- **Parameter Description**

Parameter Name	Type	Input/ Output	Description
slave_address	int16_t	Input	Slave Address
joint_angles	const std::vector<double>&	Input	The initial position of the dexterous hand during normal force control requires the input of the normalized angle values of 11 adjustable joints .
param	const ForceCtlParam&	Input	Force control parameter configuration, default parameters will be used if not set

- **Return value**

1: Configuration successful
0: Configuration failed

- **Special Instructions**

- The parameter configuration of normal force control and tangential force control will overwrite each other , and the two cannot be set and used simultaneously. After being overwritten, the parameters need to be reset before the calculation of the target position can be performed.
- When the current position of the dexterous hand is different from the initial position joint_angles where force control needs to be performed, it is necessary to wait for the dexterous hand to reach the initial position by sleeping as appropriate.

1.63 computeCommonForceTargetCommand

- **Function Overview**

In normal force control mode, calculate the target joint angle command for the next motion based on the current force sensor data.

- **Function prototype**

```
int computeCommonForceTargetCommand(int16_t slave_address, std::vector<double>& command);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
command	std::vector<double>&	Output	Normalized angle command for the next movement, containing 11 adjustablejointdata

- **Return value**

1: Calculation successful
0: Calculation failed

1.64 setTangentialForceCtlParam

- **Function Overview**

Configure **the initial position and control parameters of tangential force control** . After configuration is completed, calculate the target command for each motion through computeTangentialForceTarget Command.

- **Function prototype**

```
int setTangentialForceCtlParam(int16_t slave_address,
                               const std::vector<double>& joint_angles,
                               const TanForceCtlParam& param);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
joint_angles	const std::vector<double>&	Input	The initial position of the dexterous hand during tangential force control requires the input of the normalized angle values of 11 adjustable joints
param	const TanForceCtlParam&	Input	Tangential force control parameter configuration, default parameters will be used if not set

- **Return value**

1: Configuration successful
0: Configuration failed

- **Special Instructions**

- The parameter configuration of normal force control and tangential force control will **overwrite each other** , and the two cannot be set and used simultaneously. After being overwritten, the parameters need to be reset before the calculation of the target position can be performed.
- When the current position of the dexterous hand is different from the initial position joint_angles where force control needs to be performed, it is necessary to wait for the dexterous hand to reach the initial position by sleeping as appropriate.

1.65 computeTangentialForceTargetCommand

- **Function Overview**

In the tangential force control mode, calculate the target joint angle command for the next motion based on the current force sensor data.

- **Function prototype**

```
int computeTangentialForceTargetCommand(int16_t slave_address,
                                        std::vector<double>& command);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
command	std::vector<double>&	Output	The normalized angle command for the next movement, containing 11 adjustable joint data

- **Return value**

1: Calculation successful
0: Calculation failed

1.66 upgradeSensor

- **Function Overview**

Upgrade the sensor firmware version. Please power on the dexterous hand again before starting the upgrade.

- **Function Prototype**

```
1  UpgradeErrorCode upgradeSensor(int16_t slave_address, const std::vector<UpgradeFileAndPart>& info);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
info	const std::vector<UpgradeFileAndPart>&	Input	List of sensor firmware and upgrade components; the upgrade file must be placed in the same directory as the program

- **Description of UpgradeFileAndPart Structure**

field	Type	Description
file_path	int16_t	Firmware file path
upgrade_part	std::set<int16_t>	Set of sensor part numbers requiring upgrade

- **return value**

Returns the UpgradeErrorCode enumeration value, whose meaning is shown in the table below (the same enumeration is used in Section 1.67):

Enumeration values	Numerical Value	Description
UPGRADE_CHECK_OK	1001	Upgrade check normal
UPGRADE_FAIL	1002	Upgrade failed
UPGRADE_SUCCESS	1003	Upgrade successful
FILE_FORMAT_ERROR	2001	Abnormal file format
FILE_SIZE_ERROR	2002	Abnormal File Size
FILE_MATCH_ERROR	2003	File Matching Exception
FILE_NAME_ERROR	2004	Abnormal File Name
FILE_COUNT_ERROR	2005	Abnormal Number of Files
FILE_EXIT_ERROR	2006	File does not exist
FILE_VERSION_ERROR	2007	File version error
FILE_HAND_TYPE_ERROR	2008	The file corresponds to an incorrect type of dexterous hand
PARAM_PART_ERROR	3001	Invalid part parameter
HAND_FAULT_ERROR	4001	Equipment failure error
HAND_ENABLE_ERROR	4002	Device Enable Error
SEND_FILE_OPERATE_ERROR	5001	File operation exception
SEND_FRAME_1_ERROR	5002	First frame transmission failed
SEND_FRAME_N_ERROR	5003	Intermediate frame transmission failed
SEND_FRAME_LAST_ERROR	5004	Final frame transmission failed
SEND_EXTRA_ACTION_ERROR	5005	Action confirmation failed
FILE_SEND_SUCCESS	5006	File sent successfully
UPGRADE_REQUEST_ERROR	5007	Failed to request upgrade instruction
UPGRADE_EXECUTE_ERROR	5008	Upgrade instruction execution failed
UPGRADE_TYPE_ERROR	5009	Upgrade Type Error

1.67 upgradeSensor

- **Function Overview**

Upgrade the firmware of the dexterous hand (main control board / electric drive board).

- **Function Prototype**

```
1  UpgradeErrorCode upgradeFirmware(int16_t slave_address, const std::string& file_path);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
file_path	const std::string&	Input	Firmware file path

- **return value**

Returns an UpgradeErrorCode enumeration value, whose meaning is specified in the table of Section 1.66.

1.68 getUpgradeInfo

- **Function Overview**

Obtain the firmware upgrade progress information. If you need to use this interface, please create a separate thread to call this interface in a loop for polling before invoking upgradeFirmware.

- **Function Prototype**

```
1  const std::vector<UpgradeProcess> getUpgradeInfo(int16_t slave_address);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

- **return value**

Returns a list of UpgradeProcess, where each element contains:

field	Type	Description
upgrade_type	FirmwareUpgradeType	Upgrade type (enumeration, see the table below)
part	list[int]	Upgraded firmware component number
process_data	int	Upgrade progress value, range [0,100]
status	std::string	Upgrade Status Description

- **FirmwareUpgradeType Enumeration Values**

Enumeration values	Numerical Value	Description
INVALID_UPGRADE	0	Invalid upgrade type
MAINBOARD_UPGRADE	1	Main Control Board Upgrade
DRIVER_UPGRADE	2	Electric drive board upgrade
SENSOR_UPGRADE	3	Sensor Upgrade

2. Camera control class DexH15Camera

is responsible for the connection, image acquisition, and parameter configuration of the camera equipment supporting the dexterous hand.

2.1 connectCameraDevice

- **Function Overview**

  Open and connect to the specified camera device.

- **Function prototype**

```
1  void connectCameraDevice(const std::string& camera_port_num);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
camera_port_num	const std::string&	Input	Camera Device Number

- **Return value**

  No return value (void). Throws an exception when the connection fails.

2.2 releaseCameraDevice

- **Function Overview**

  Release camera device resources and disconnect.

- **Function prototype**

```
1  void releaseCameraDevice();
```

- **Parameter Description**

  None

- **Return value**

  No return value (void).

2.3 getFrame

- **Function Overview**

  Capture the current frame image from the camera.

- **Function prototype**

```
1  cv::Mat getFrame();
```

- **Parameter Description**

  None

- **Return value**

  Returns an image frame of type cv::Mat, with a default resolution of 640×480 and a frame rate of 30 frames per second. If the camera is not opened, it returns an empty Mat.

2.4 showImage

- **Function Overview**

  Displays the specified image frame in the window.

- **Function prototype**

```
1  void showImage(const std::string& imgName, const cv::Mat& img);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
imgName	const std::string&	Input	Display the name of the window
img	const cv::Mat&	Input	Image to be displayed

- **Return value**

  No return value (void).

2.5 setCameraConfig

- **Function Overview**

  Set the resolution and frame rate parameters of the camera.

- **Function prototype**

```
bool setCameraConfig(int width, int height, float fps);
```

- **Parameter Description**

Parameter Name	Type	Input/Output	Description
width	int	Input	Image resolution width (pixels)
height	int	Input	Image resolution height (pixels)
fps	float	Input	Camera frame rate (frames per second)

- **Return value**

  true: Configuration successful
  false: Configuration failed

- **Camera Parameter Description**

  Set camera parameters, including resolution and frame rate. Only 640x480, 1280x720, and 1920x1080 resolutions are supported. The frame rate only supports 30fps for 640x480 and 1280x720 resolutions, and 60fps for 1920x1080 resolution.

3. Kinematic Analysis Class DexH15Kinematic

is responsible for the forward and inverse kinematic transformation between the dexterous hand joint angles and motor positions, as well as the mutual conversion between normalized angles and real angles.

3.1 getSDKVersion

- **Function Overview**

  Get the version number of the current SDK.

- **Function prototype**

```
const std::string getSDKVersion();
```

- **Parameter Description**

  None

- **Return value**

  Returns the SDK version number string (std::string).

3.2 inverseKinematicParse

• Function Overview

Inverse Kinematics Analysis : Input the real angle values of 11 adjustable joints, calculate and output the corresponding 7 motor position values.

• Function prototype

```
int inverseKinematicParse(const HandType hand_type,
                          const std::vector<double>& real_angle,
                          std::vector<int16_t>& motor_pos,
                          const std::string& hardware_version = "");
```

• Parameter Description

Parameter Name	Type	Input/Output	Description
hand_type	const HandType	Input	Left or right hand type
real_angle	const std::vector<double>&	Input	Set of real angle values for 11 adjustablejoints (unit: °)
motor_pos	std::vector<int16_t>&	Output	7 motor position information obtained through parsing
hardware_version	const std::string&	Input	Hardware version (optional, default is empty)

• Description of real_angle element

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	corresponds to the finger	True Angle Range (°)	Description
0	index_joint1	index finger	[0, 80]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 80]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 80]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 80]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 80]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 80]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 80]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 80]	End joint, bending and extension
8	thumb_joint1	Thumb	[0, 109.6]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 87]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 120.1]	End joint, bending and extension

• Return value

1: Parsing successful
0: Parsing failed

3.3 forwardKinematicParse

• Function Overview

Forward kinematics analysis: Forward Kinematics Analysis: Input 7 motor position values and 8 magnetic encoder data, then calculate and output the actual joint angle values. When return_joint_flag is false, 11 adjustable joint angles are output; when it is true, 4 additional distal interphalangeal (DIP) joints are output, totaling 15 joint angles.

• Function prototype

```
int forwardKinematicParse(const HandType hand_type,
                          const std::vector<int16_t>& motor_pos,
                          const std::vector<int16_t>& magnetic_encoder,
                          std::vector<double>& real_angle,
                          const std::string& hardware_version = "");
                          bool return_joint_flag = false);
```

• Parameter Description

Parameter Name	Type	Input/Output	Description
hand_type	const HandType	Input	Left or right hand type
motor_pos	const std::vector<int16_t>&	Input	Set of 7 motor position values, with a range of [0, 10000]
magnetic_encoder	const std::vector<int16_t>&	Input	8 sets of magnetic encoder data
real_angle	std::vector<double>&	Output	The actual joint angle values obtained through parsing (unit: °), the number of elements is determined by return_joint_flag
hardware_version	const std::string&	Input	Hardware version (optional, defaults to empty)
return_joint_flag	bool	Input	A flag that controls the number of returned joints, which defaults to false (returning 11 adjustable joints; when set to true, it returns 15 joints, i. e., 11 adjustable joints plus 4 distal interphalangeal (DIP) joints at the finger ends)

• Return value

1: Parsing successful
0: Parsing failed

3.4 calculateRealAngle (Kinematics Class)

• Function Overview

Calculate the corresponding real physical angles (unit: degrees) based on the normalized angle set of 11 adjustablejoints.

• Function prototype

```
int calculateRealAngle(const std::vector<double>& normalize_angle,
                       std::vector<double>& real_angle,
                       const std::string& hardware_version = "");
```

• Parameter Description

Parameter Name	Type	Input/Output	Description
normalize_angle	const std::vector<double>&	Input	Normalized angle set of 11 joints
real_angle	std::vector<double>&	Output	The calculated set of 11 real angles (unit: °)
hardware_version	const std::string&	Input	Hardware version (optional, default is empty)

• Description of normalize_angle element

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	corresponds to the finger	Normalized Input Range	Description
0	index_joint1	index finger	[0, 1000]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 1000]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 1000]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 1000]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 1000]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 1000]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 1000]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 1000]	End joint, bending and extension
8	thumb_joint1	Thumb	[-500, 500]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 1000]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 1000]	End joint, bending and extension

• Return value

1: Calculation successful
0: Calculation failed

3.5 calculateNormalizeAngle (Kinematics Class)

• Function Overview

Calculate the corresponding set of normalized angles based on the real physical angles of the 11 adjustable joints.

• Function prototype

```
int calculateNormalizeAngle(const std::vector<double>& real_angle,
                            std::vector<double>& normalize_angle,
                            const std::string& hardware_version = "");
```

• Parameter Description

Parameter Name	Type	Input/Output	Description
real_angle	const std::vector<double>&	Input	Set of real angles of 11 joints (unit:°)
normalize_angle	std::vector<double>&	Output	The set of 11 normalized angles obtained from the calculation
hardware_version	const std::string&	Input	Hardware version (optional, default is empty)

• Description of real_angle element

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	corresponds to the finger	True Angle Range (°)	Description
0	index_joint1	index finger	[0, 80]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 80]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 80]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 80]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 80]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 80]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 80]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 80]	End joint, bending and extension
8	thumb_joint1	Thumb	[0, 109.6]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 87]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 120.1]	End joint, bending and extension

• Return value

1: Calculation successful
0: Calculation failed

PaXini



PaXini Tech

Tel: **PaXini Technology (Shenzhen) Co., Ltd.** : +86 755 2357 4593

PaXini Technology (Shanghai) Co., Ltd. : +86 21 5456 1536

PaXini Tianjin Super EID Factory : +86 22 8241 1882

Email: **sales_global@paxini.com**

Website: **www.paxini.com**

Add: **PaXini Technology (Shenzhen) Co., Ltd.**
34-35th Floor，Qiancheng Business Center, 5Haicheng Road, Bao'an District, Shenzhen City, Guangdong Province

PaXini Technology Intelligent & Manufacture Laboratory
Yongfeng Community, Baoan District, Shenzhen City, Guangdong Province

PaXini Technology (Shanghai) Co., Ltd.
Room 207-209, Block A, No. 391 Guiping Road, Xuhui District, Shanghai

PaXini (Tianjin) Super EID Factory
Kongtian Digital Industry Park, Hexi District, Tianjin

PaXini Manufacture and Development Center (Suzhou)
3rd Floor, Building A02, No. 1202, Fuxin Road, Yangshe Town, Zhangjiagang City, Suzhou City, Jiangsu Province

PaXini Sensing Technology Research Branch (Shenzhen)
Cloud City, Building 1, Phase 6 Wangkeyuncheng, Nanshan District, Shenzhen Guangdong Province

Scan for more information

AliExpress

Amazon